

Independent Academic Path Optimizer

API References



AI-Assisted Class Scheduling Application

Document Type	API References
Project	Independent Academic Path Optimizer
Course Deliverable	CS691 Sprint 2 Final Application
Version	v1.0.0
App URL	https://2026-paceu-capstone.github.io/intelligent-academic-path-optimizer
Prepared Date	August 2026

ROUTES:

Common HTTP Responses:

- 200 OK — Request succeeded
- 201 Created — New resource successfully created
- 400 Bad Request — Missing or malformed input
- 401 Unauthorized — No valid session/authentication
- 404 Not Found — Resource doesn't exist
- 409 Conflict — Resource already exists
- 422 Unprocessable Entity — Data is well-formed but invalid
- 500 Internal Server Error — Unexpected server failure
- 503 Service Unavailable — Dependency down or not configured

Backend:

1. Session Verification

GET

[/auth](#)

Sends a request with the user's cookies containing their email and session ID to confirm if the user has a valid session. This allows them to automatically login on the same browser if they closed the site or returned to it within 7 days.

If there is a valid session, they will receive their user information.

Example Request:

```
GET https://<username>.pythonanywhere.com/auth
{
  user: "user_email",
  session_id: "sessionID"
}
```

Response:

```
{
  "email" : "userEmail",
  "name" : "name",
  "academic" : {},
  "schedule" : {},
  "transcript" : {}
}
```

2. Email Availability Check

POST

[/auth/email](#)

Used during signup for confirming if a user account already exists, so that the same email can't be used for two different accounts.

Example Request:

```
POST https://<username>.pythonanywhere.com/auth/email
{
  user: "user_email",
}
```

Response:

```
{
  "exists" : True,
  "message" : "Email available"
}
```

3. Create Account

POST

[/signUp](#)

Used for creating the user account. If successful returns a copy of the user information and their session cookie.

Example Request:

```
POST https://<username>.pythonanywhere.com/signUp
{
  user: "user_email",
  password: "password",
  name: "name",
  degreeLevel: "Undergraduate",
  major: "Computer Science",
  startingSemester: "Spring 2024I",
  endingSemester: "Fall 2025",
  transcript: {},
  chat: "More AI courses",
}
```

Response:

```
{
  "user" : "userEmail"
  "sessionID" : "sessionID"
  "email" : "userEmail",
  "name" : "name",
  "academic" : {},
  "schedule" : {},
}
```

```
"transcript" : {}  
}
```

4. Log Into Account

POST

[/signIn](#)

Used for logging into an existing account. If successful returns a copy of the user information and their session cookie.

Example Request:

```
POST https://<username>.pythonanywhere.com/signIn  
{  
  user: "user_email",  
  password: "password",  
}
```

Response:

```
{  
  "user" : "userEmail"  
  "sessionID" : "sessionID"  
  "email" : "userEmail",  
  "name" : "name",  
  "academic" : {},  
  "schedule" : {},  
  "transcript" : {}  
}
```

5. Sign Out Account

POST

[/signOut](#)

Signs the user out by deleting the current session cookies from the user and the database, requiring them to log back in to use IAPO.

Example Request:

```
POST https://<username>.pythonanywhere.com/signOut  
{  
  session_id : "sessionID"  
}
```

Response:

```
{
```

```
"success" : True
}
```

6. Update Credentials

POST

[/updateUser](#)

Used for updating user credentials (Name, email, password). Returns a copy of the user information.

Example Request:

POST <https://<username>.pythonanywhere.com/updateUser>

```
{
  user: "user_email",
  password: "password",
  name: "name",
}
```

Response:

```
{
  "email" : "userEmail",
  "name" : "name",
  "academic" : {},
  "schedule" : {},
  "transcript" : {}
}
```

7. Update Academic Details

POST

[/updateUser/academic](#)

Used for updating user academic details (transcript, major, credits per semester, schedule, etc.) Returns a copy of the user information

Example Request:

POST <https://<username>.pythonanywhere.com/updateUser/academic>

```
{
  email: "userEmail",
  degreeLevel: "Undergraduate",
  major: "Computer Science",
  startingSemester: "Spring 2024",
  endingSemester: "Fall 2025",
  transcript: {},
  chat: "More AI courses",
}
```

Response:

```
{
```

```
"email" : "userEmail",
"name" : "name",
"academic" : {},
"schedule" : {},
"transcript" : {}
}
```

8. Program Information

GET

</api/info>

Gets the current program info for available majors and semesters

Example Request:

GET https://<username>.pythonanywhere.com/api/info

Response:

```
{
  "semesters": {},
  "majors": {},
}
```

AI:

9. Service Status Check

GET

</api/health>

Confirms the service is running, whether the Gemini API key is configured, and if the course catalog is successfully loaded

Example Request:

GET https://<username>.pythonanywhere.com/api/health

Response:

Status: 200 OK

```
{
  "status": "ok",
  "mode": "real",
  "catalog_loaded": true
  "catalog" : {}
  "catalog_summary" : {}
}
```

10. Recommendation Extraction

POST

[/api /extract](#)

Accepts the frontend field data and returns a JSON object containing course plan recommendations to be handed to the solver

Example Request:

POST <https://<username>.pythonanywhere.com/api/extract>

```
{
  degreeLevel: "Undergraduate",
  major: "Computer Science",
  startingSemester: "Spring 2024",
  endingSemester: "Fall 2025",
  transcript: {},
  chat: "More AI courses",
}
```

Response:

Status: 200 OK

```
{
  "tabs": [
    {
      "id": "completed",
      "title": "Completed Courses",
      "completedCourses": ["CS505", "CS604", "CS608", "CS610", "CS612", "CS623"],
      "waivedOrTransferred": [],
      "inProgressCourses": [],
      "notes": [
        "candidate_plans are suggestions; the solver validates them.",
        "Optimization goal defaulted to 'balanced' because not specified."
      ]
    },
    {
      "id": "schedule",
      "title": "Recommended Schedule"
    }
  ],
  "selectedPlan": {
    "name": "AI-focused, front-loaded",
    "score": 0.82,
    "rationale": "Takes your best-fit electives as early as prerequisites allow, capstone last.",
    "honoredAspects": ["no Friday", "online-preferred"]
  },
  "schedule": [
    {
      "term": "Spring2027",
    }
  ]
}
```

```

    "courses": [
      {"course": "CS627", "name": "Artificial Intelligence", "score": 0.93, "credits": 3, "rationale":
"covers AI fundamentals"},
      {"course": "CS661", "name": "Python Programming", "score": 0.71, "credits": 3, "rationale":
"supports ML work"}
    ],
    "credits": 6
  },
  {
    "term": "Fall2027",
    "courses": [
      {"course": "CS672", "name": "Intro to Deep Learning", "score": 0.95, "credits": 3, "rationale":
"fits your AI Engineer goal"},
      {"course": "CS617", "name": "Game Programming", "score": 0.20, "credits": 3, "rationale":
"limited overlap with your interests"}
    ],
    "credits": 6
  },
  {
    "term": "Spring2028",
    "courses": [
      {"course": "CS691", "name": "Computer Science Project", "score": 0, "credits": 3,
"rationale": "Required capstone, taken at the end of the program."}
    ],
    "credits": 3
  }
]
}

```

11. Transcript Verification

POST

</api/transcriptCheck>

Accepts a PDF that is checked by Gemini to see if it is a valid transcript or not

Example Request:

```

POST https://<username>.pythonanywhere.com/api/transcriptCheck
{
  transcript: {},
}

```

Response:

Status: 200 OK

```

{
  "is_transcript": true,
  "confidence": 0.92,
  "reason": "Document contains course codes, grades, credits, and semester labels consistent

```


with an academic transcript."
}